laga.

Lease Agreement Guard Agent

BACKEND IMPLEMENTATION & HANDOFF

One clear path from API contract to connected app.

An implementation handbook for backend engineers, integration testers and deployment owners. Includes the complete endpoint and DTO reference.

CONTRACT	PREPARED
1.1.0 · OpenAPI 3.1.0	**October 31, 2026**
INITIAL JURISDICTION	INTEGRATION SURFACE
North Carolina · NC	**11 endpoints · 26 models**

Start with the implementation route. Use the endpoint pages while building, the field dictionary while mapping DTOs, and the acceptance matrix before switching the application to API mode.

This is an integration target and engineering handoff, not a deployed backend or a legal compliance certification.

NAVIGATION

Find what you need.

Read the implementation chapters in order on the first pass. Return to the endpoint and model reference for exact wire details. Section links are clickable.

IMPLEMENTATION

01 / Start here: the integration route 4
02 / Files, ownership and environments 5
03 / Decide how the API is reached 6
04 / HTTP, sessions and error envelopes 7
05 / Dates, identities and DTO conventions 8
06 / Test in the API Explorer 9
07 / Connect in development 11
08 / Deploy behind a same-origin route 13
09 / Deploy with direct cross-origin calls 15
10 / Endpoint map and implementation order 16
11 / Policy storage, versions and async processing 30
12 / NC state and compliance behavior 32
13 / Exact review metadata and hashing 33
14 / Reviewer integration and safe clarification 34
15 / Manual HTTP walkthrough: curl 36
16 / Manual HTTP walkthrough: PowerShell 38
17 / Acceptance matrix and test evidence 40
18 / Production cutover and rollback 41
19 / Troubleshooting by symptom 42
20 / Decisions to close before implementation 44

RELEASE REFERENCE

21 / References, version and handoff receipt 72

ENDPOINT REFERENCE

E01 / POST /auth/login 17
E02 / GET /auth/session 18
E03 / POST /auth/logout 19
E04 / GET /workspace 20
E05 / GET /leases/{id} 21
E06 / POST /assistant/messages 22
E07 / POST /requests 24
E08 / POST /requests/{id}/reply 25
E09 / POST /policies 26
E10 / DELETE /policies/{id} 28
E11 / POST /properties 29

COMPLETE DATA DICTIONARY

A1 / Identity and session models 45
A2 / Properties and workspace 47
A3 / Leases, details and question scope 50
A4 / Policy upload and immutable versions 52
A5 / Assistant requests and evidence 55
A6 / Clarification request workflow 58
A7 / Compliance review metadata 60
A8 / Errors and mutation acknowledgments 62

COPYABLE RESPONSE REFERENCE

Examples you can map directly to your service.

The JSON appendix contains complete response envelopes. It complements the route-specific request samples and the field dictionary.

WORKED JSON RESPONSES

J / anonymousSession — Session 64
J / authenticatedSession — Session 65
J / registeredProperty — Property 66
J / emptyLandlordWorkspace — Workspace 67
J / pendingPolicy — PolicyVersion 68
J / clarificationAnswer — Answer 69
J / syntheticReviewedAnswer — Answer 70
J / versionConflict — Error 71

How to read the examples

Every ID, account and date is fictional. Copy the structure, not the sample authorization or verdict.

For the live backend, replace IDs with records returned by the current workspace and use a real test account for each role.

The passed-review example is synthetic. Its hash matches the example text only, its law revision is a placeholder, and its timestamps expire.

Two different sample files

`contracts/integration-examples.json` contains request fixtures. `docs/integration-guide-examples.json` contains the response examples printed in this handbook.

Required means present

Keep required nullable fields present with null when unknown. Omit optional fields only when permitted. Extra undeclared object fields are rejected by most DTO schemas.

Use the correct example for the task

An empty landlord workspace is a valid bootstrap shape, not a substitute for a failed database read. A processing policy is durable accepted metadata, not ready evidence. A clarification answer is a safe supported workflow outcome when a legal check is missing; do not include unreviewed advice in its text.

> Validate examples and real responses with the same OpenAPI version. The machine-readable spec remains authoritative for fields and envelopes; the implementation chapters explain ownership, date boundaries and security requirements that schemas cannot prove.

IMPLEMENTATION

01 / Start here: the integration route

This handbook is for backend engineers, integration testers and the person deploying Laga. It describes the contract the existing frontend consumes. Backend language and framework are your choice; keep the HTTP behavior and DTOs consistent.

Read this first

1. Use contracts/openapi.json, version 1.1.0, as the machine-readable API definition. This PDF explains behavior and includes its complete field reference.
2. Build the 11 routes under /api/v1. Use the response envelopes exactly; do not return raw database objects.
3. Test independently in the API Explorer. Then test the application in API mode with both roles. Those are separate acceptance steps.
4. Choose a deployment topology. The backend's physical server does not need to be the frontend's server.
5. Connect by configuration, restart or rebuild, and run the acceptance matrix in this handbook. A conforming backend should not require React component changes.

One-minute navigation

- Application: http://localhost:5174 after starting the local development server.
- Explorer: turn on Developer Tools on the login page or home dashboard, then choose API Docs. The toggle is not shown on lease, conversation or document pages.
- Direct explorer address: http://localhost:5174/api-docs/.
- Standalone kit: extract laga-api-testing-kit-v1.1.0.zip and run node scripts/serve.mjs from its root; no npm installation is required for the built kit.
- Full repository: Node.js 22+, npm ci, then npm run dev. On PowerShell, use npm.cmd if npm.ps1 is blocked.

What a successful handoff includes

A backend URL, implemented contract version, provisioned test accounts, representative IDs, documented hosting/auth settings, exported explorer results, backend test evidence and end-to-end application sign-off. A green response-schema check alone is insufficient.

Status of the software

The UI and explorer exist. The current default is a fictional mock workspace. The reusable Python law-cache/reviewer boundary exists, but authentication, durable data, real PDF ingestion, retrieval/model execution and a qualified legal reviewer still need production backend implementation. This PDF does not certify any uploaded lease or policy as lawful.

IMPLEMENTATION

02 / Files, ownership and environments

Source-of-truth order

Use the OpenAPI document for transport shapes and paths, the behavioral contract for authorization/timelines/review invariants, and the frontend adapter for the exact browser call behavior. If they disagree, resolve the discrepancy before implementation; do not silently introduce a second API shape.

File or artifact	Purpose
contracts/openapi.json	OpenAPI 3.1; all routes, JSON schemas, authentication declarations and documented responses.
contracts/examples.json	Complete fictional session/workspace/lease/answer envelopes for comparison.
contracts/integration-examples.json	Request fixtures and a synthetic passed-review answer; timestamps expire.
frontend/src/services/httpGateway.js	Application HTTP adapter: credentials, CSRF, envelope handling and answer-review display guard.
frontend/src/lib/compliance.js and policies.js	Exact review identity/freshness checks used by the frontend.
frontend/src/api-docs/	Independent request tester generated from the shared spec; not a live backend.
src/laga/compliance.py	Reusable SQLite source cache and default-deny reviewer boundary; not an API service.
docs/BACKEND_CONNECTION.md	Editable connection/deployment instructions.
docs/INTERFACE_CONTRACT.md	Behavioral implementation agreement.

Division of work

The backend owns sessions, role/organization authorization, data persistence, private files, ingestion, effective-date filtering, review implementation, model calls, deterministic calculations and audit records. The frontend owns rendering and browser request mapping. The deployment owner owns HTTPS, routing, cookie/origin configuration, CORS where necessary and the correct environment at build time.

Environments are explicit

Use separate development/staging/production accounts, storage and secrets. Example IDs and fictional credentials are not real seeded records. Record the backend base URL and contract version in every test report. The explorer's target and the application's API configuration are independent.

IMPLEMENTATION

03 / Decide how the API is reached

The physical server location and the browser's origin are separate choices. An origin is scheme + hostname + port; two services on the same machine but different ports are cross-origin.

Arrangement	Browser calls	Application setting	Requirements
UI and API served under the same origin	`https://app.example.com/api/v1/...`	`/api/v1`	Route `/api` to the backend; no browser CORS needed
Backend on another server, reverse-proxied through UI origin	`https://app.example.com/api/v1/...`	`/api/v1`	Reverse proxy reaches the remote backend; preserve path and cookie behavior
Browser calls a separate API origin directly	`https://api.example.com/api/v1/...`	Full API URL	Credentialed CORS, HTTPS, compatible cookies and CSRF origin validation

The reverse-proxy arrangement works whether the backend is local, in another container, or on a different server. It also avoids dependence on cross-site browser cookies. Use direct cross-origin hosting when your authentication and network configuration support it.

The integration boundary

Browser → /api/v1 route → your HTTP service → authorized data/files → ingestion/review/retrieval workers. Worker-to-worker protocols and database schemas are internal implementation choices; the UI consumes the HTTP DTOs described here.

Recommended default while hosting is undecided

Keep the browser-facing API base as /api/v1 and use a reverse proxy. This works with a backend on the same machine, another container, or another server. If the browser must call the API origin directly, use the full HTTPS API base and configure the cross-origin requirements in chapter 08.

Prefix examples

- Correct explorer/application base: https://api.example.com/api/v1.
- Correct development proxy origin: https://staging-api.example.com.
- Incorrect proxy target: https://staging-api.example.com/api/v1 when the incoming path already contains /api/v1. This can duplicate the prefix.
- The /api-docs/ page is frontend content; it must not be forwarded by a broad /api prefix rule.

IMPLEMENTATION

04 / HTTP, sessions and error envelopes

Success responses are `{ "data": <DTO> }`, including logout and DELETE. Do not return an empty 204 response. Failures use `{ "error": { "code": "VERSION_CONFLICT", "message": "This policy changed. Refresh the library before adding a version.", "requestId": "req-unique", "fields": {} } }`. IDs are opaque stable strings; never derive authorization from their format.

GET /auth/session returns a Session; anonymous visitors receive user:null and csrfToken:null, not 401. POST /auth/login receives email/password and returns Session. The UI's customer/landlord selector is a presentation hint: the server determines the role and permissions. Use Secure, HttpOnly, SameSite cookies in production, session rotation at login, CSRF protection (including login via origin checks or a pre-auth token), password hashing, rate limiting and generic invalid-login messages. Never return password hashes or accept client-supplied permissions. POST /auth/logout invalidates the session and returns data:{signedOut:true}. Authenticated mutations carry X-CSRF-Token from Session. The frontend sends credentials:include. 401 clears the visible workspace; 403 preserves it and reports the denied action. Tenant requests for foreign objects should return 404 to avoid disclosing their existence.

Return safe messages, field-level validation details and a requestId. Never send stack traces or internal storage paths. 400 malformed input, 401 unauthenticated, 403 forbidden, 404 invisible/missing object, 409 state/version conflict, 413 file too large, 422 validation failure, 429 rate limit, 503 unavailable. JSON schemas describe DTOs; enforce lengths, enums, ownership and file validation server-side too.

Compatibility details that often break integrations

- Creation endpoints return HTTP 200, not 201, under this contract. Logout and cancellation return JSON envelopes, not 204.
- Use Content-Type: application/json on JSON responses, including errors. The explorer validates the media type and the documented status-specific envelope.
- The OpenAPI security scheme names the cookie laga_session. Keep its content opaque and server-managed.
- Origin validation on login is compatible with the current frontend. An extra pre-authentication token exchange or header not in this contract requires a coordinated change.
- The application sends {} on logout; the explorer sends no body because no request body is specified. Accept both without requiring extra fields.
- A 401 is a sign-in failure/session-expiry path. A 403 is an authenticated denial, and should not erase a valid session. A foreign tenant object should ordinarily be hidden with 404.
- The API must authorize data before returning it. Hiding a menu or toggling Developer Tools is not authorization.

IMPLEMENTATION

05 / Dates, identities and DTO conventions

Stable identities and units

IDs are opaque strings. Persist relationships explicitly: tenant → authorized lease → property → state. Bind all landlord records to the authenticated organization internally; an organization ID need not be accepted from the browser. A URL ID or thread ID never grants access.

Money is integer USD cents throughout DTOs and calculation inputs/results. For example, 220000 means USD 2,200.00. Do not return a formatted currency string where an integer is required. Unknown values must not become zero silently.

Dates have two distinct meanings

Calendar dates use YYYY-MM-DD, such as effectiveFrom and asOf. Validate real dates, not only the regex: 2026-02-30 is invalid. Instants such as uploadedAt, createdAt, checkedAt and expiresAt carry offsets; UTC Z is a good interoperable choice.

Workspace.timeZone is an IANA zone. For an NC property, configure America/New_York as appropriate to the real property/business rules. Persist the policy family's time zone so old intervals do not change when display settings change. The demo currently exposes America/Chicago and a frozen asOf of 2026-10-30; those are fixture values, not deployment defaults or today's business date.

Policy intervals are half-open: effectiveFrom <= asOf and (effectiveTo is null or asOf < effectiveTo). A predecessor ending 2026-11-15 is excluded on November 15. Its replacement starts that day.

Required, optional and nullable are different

If a required field is unknown and its type allows null, include it with null. Do not omit it. Optional fields may be omitted, but cannot be null unless the type permits it. Most object schemas set additionalProperties: false; do not append debug, database or model fields to them.

Small-portfolio contract

Workspace returns the current authorized collection without pagination. Its summary must be calculated from the same authorized lease/property set. Avoid double-counting shared tenants as occupied units. Empty arrays are valid, but the backend must define intentional behavior for a tenant without a provisioned lease; the home page shows a getting-ready empty state when no lease is assigned; provision the association before testing lease-dependent flows.

Property.style is a required UI illustration key: willow, juniper or cedar. Supply a supported default even if your database has no illustration concept. User.role is tenant or landlord; Customer is a UI label for tenant. Permissions are exact strings, not display labels.

IMPLEMENTATION

06 / Test in the API Explorer

Entry from the user interface

On the login page or a signed-in home dashboard, enable Developer Tools. API Docs appears in its reserved row and opens the explorer in another tab. This does not change your role, backend permissions or the application's mock/API mode. The toggle does not appear on other workspace pages. It resets when the page reloads.

1. Set **Backend API base URL** to `/api/v1` for a same-origin route or development proxy, or to a full address such as `https://api.example.com/api/v1`. Select **Apply server**. Include the version prefix exactly once; endpoint paths are appended to it.
2. Send `GET /auth/session`. An anonymous session returns HTTP 200 with `data.user: null` and `data.csrfToken: null`.
3. Select `POST /auth/login`, replace the fictional credentials with a test account, and send. The backend sets its session cookie. A successful schema-valid response supplies the CSRF token, which the explorer captures automatically. The browser handles HttpOnly cookies; JavaScript cannot read them.
4. Test `GET /workspace` and a lease visible to that user. Copy real IDs from your responses into path parameters and JSON bodies. The example IDs are placeholders, not seeded backend records.
5. With a landlord account, register an NC property, upload a PDF, and test a new policy version using the current family ID and `expectedVersion`. Multipart requests let the browser set the boundary. Uploaded PDFs must be nonempty and at most 10 MiB; server-side type/content validation remains required.
6. Test clarification requests with a tenant account, landlord replies, and assistant responses with state-law review metadata. Use a test session for each role and log out between accounts.
7. Inspect HTTP status, response body, visible headers and schema errors. An error envelope may match the schema while the operation still fails. Export the test report for the integration handoff; it excludes bodies, passwords and CSRF tokens. The base URL, method, path template, HTTP status and schema check are included.

No requests execute merely by opening the explorer. Mutation endpoints act on the selected backend when **Send request** is chosen. Requests can be canceled and time out after 30 seconds. Canceling the browser request does not undo work already committed by the backend; check the workspace before retrying a mutation.

CSRF tokens, credentials and response data are held in page memory, not local/session storage. Reloading clears them. Changing the applied server clears captured authentication metadata, results and checks; it does not log out the old server or delete its cookies. Network errors may come from CORS, mixed content, certificates or an unavailable server; browsers do not expose detailed CORS failures to JavaScript.

The explorer is a developer tool. Publish it only to environments/audiences intended for testing, or restrict `/api-docs/` at your web server. It is not a replacement for backend authorization.

What a green result means

IMPLEMENTATION

06 / Test in the API Explorer / continued

It means the actual HTTP status is documented, the body is JSON with the expected media type, and its shape matches that response schema. It does not establish authorization, correct arithmetic, legal validity or the completeness of a government source. An HTTP 401 can have a correctly shaped error envelope; that is not a successful endpoint test.

Tester versus application

The explorer performs full schema checks using AJV with format checks disabled. Calendar dates and timestamp semantics still require server validation. The application adapter performs envelope/error handling and additional answer-review checks, but is not a full DTO validator. Do not rely on the browser to enforce backend correctness.

The explorer rejects redirects. Use final API routes that return JSON directly; do not redirect API callers to HTML sign-in pages. The application adapter has no explicit timeout, while explorer calls time out after 30 seconds. Do not use the explorer's timeout as a backend job-completion guarantee.

IMPLEMENTATION

07 / Connect in development

Copy `frontend/.env.example` to `frontend/.env.local`:

dotenv

```
VITE_LAGA_MODE=api
VITE_LAGA_API_BASE=/api/v1
```

Vite proxies `/api` to `http://127.0.0.1:8001` by default, preserving `/api/v1/...`. Your backend must serve that prefix.

To proxy to another development server, set `LAGA_API_PROXY` in the shell that starts Vite:

powershell

```
$env:LAGA_API_PROXY='https://staging-api.example.com'
npm.cmd run dev
```

sh

```
LAGA_API_PROXY=https://staging-api.example.com npm run dev
```

Use only the upstream origin for `LAGA_API_PROXY`, not `/api/v1`; Vite forwards the full incoming path. Use valid TLS certificates. The development proxy translates cookie domain/path attributes to the UI origin and `/` for local testing; do not configure Domain on host-only production cookies merely for Vite.

Alternatively bypass the proxy and set:

dotenv

```
VITE_LAGA_MODE=api
VITE_LAGA_API_BASE=https://staging-api.example.com/api/v1
```

That second configuration requires the backend to allow the actual frontend origin, typically `http://localhost:5174`. HTTPS API cookies and browser third-party cookie restrictions may prevent cross-site session testing from localhost; use the development proxy or a same-site HTTPS test environment if needed. Restart Vite after changes to `.env` or shell configuration. The explorer can test an API while the application remains in mock mode.

Reproducible repository startup

From the repository root, run npm ci followed by npm run dev. Python 3.11+ is required for the reference compliance module and its tests, not for serving the built static testing kit. Your backend framework may have its own requirements.

IMPLEMENTATION

07 / Connect in development / continued

Change only configuration for a conforming API

Set VITE_LAGA_MODE=api. Keep VITE_LAGA_API_BASE=/api/v1 when using the development proxy; use a full API URL only for direct browser access. No VITE_* variable may contain a service secret. LAGA_API_PROXY is a development-server setting, not a deployed browser setting. Restart the Vite process after changing it.

IMPLEMENTATION

08 / Deploy behind a same-origin route

Set `VITE_LAGA_MODE=api` and `VITE_LAGA_API_BASE` in the environment used for `npm run build`. Deploy the generated `dist/` directory. Vite embeds these settings during the build; changing server environment variables after serving a build does not change that build's API target. Rebuild/redeploy to change it. Never put secrets in `VITE_*` variables.

For a same-origin setup, a reverse proxy can serve the built UI and route `/api/` to a backend. This Nginx example preserves the `/api/v1/...` path:

nginx

```
server {
    listen 443 ssl;
    server_name app.example.com;
    # Configure your certificate/key and standard TLS settings here.
    root /srv/laga/dist;

    location /api/ {
        # No trailing URI slash: preserve the original /api/v1/... path.
        proxy_pass http://backend.internal:8001;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        client_max_body_size 12m;
    }

    location /api-docs/ {
        try_files $uri $uri/ =404;
        # Apply your organization's developer-only access rules here if hosted.
    }

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

The backend must trust forwarded headers only from known proxies. Configure host/origin checks and host-only cookie attributes for the browser-visible application origin. If proxying an HTTPS upstream, configure upstream certificate verification, SNI and the expected upstream Host explicitly with your hosting team. The example uses an internal HTTP upstream, not a complete production infrastructure configuration.

The included `npm start` static server serves `/api-docs/` and the built UI, but **does not proxy or implement `/api`**. Front it with a reverse proxy for same-origin API mode, or use a full cross-origin API URL at build time.

Verify routing before testing business logic

GET /api/v1/auth/session must reach the API and return its JSON Session envelope. GET /api-docs/ must reach frontend assets. The SPA fallback must not convert an API 404 into index.html. Preserve the API prefix; preserve usable cookie attributes and configure forwarded-header trust deliberately.

IMPLEMENTATION

08 / Deploy behind a same-origin route / continued

Internal service authentication

If a deployment gateway or private upstream requires a service credential, keep it in the proxy/service environment. Do not place it in frontend assets or add it to a public VITE_* variable. Gateway-specific requirements must be documented by the hosting owner.

IMPLEMENTATION

09 / Deploy with direct cross-origin calls

For a frontend at `https://app.example.com` calling `https://api.example.com`, return these headers on allowed actual responses, including error responses:

http

```
Access-Control-Allow-Origin: https://app.example.com
Access-Control-Allow-Credentials: true
Vary: Origin
```

For allowed preflight requests, respond without requiring an authenticated session:

http

```
Access-Control-Allow-Origin: https://app.example.com
Access-Control-Allow-Credentials: true
Access-Control-Allow-Methods: GET, POST, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, X-CSRF-Token
Vary: Origin
```

Use an explicit allowlist of frontend/tester origins, including scheme and port. Do not reflect arbitrary origins or use `Access-Control-Allow-Origin: *` with credentials. If using an HTTP request-ID header, expose it explicitly with `Access-Control-Expose-Headers`; the JSON error `requestId` remains available without this. Cookies belong to the API host in this topology. Keep them HttpOnly and Secure in production; choose a Path that covers the API.

Cross-origin is not always cross-site: HTTPS subdomains of the same registrable domain are normally same-site. Truly cross-site session cookies generally require `SameSite=None; Secure`, and browser third-party cookie restrictions can still block them even with valid CORS. Do not disable CSRF protection to make CORS work. Allow the intended frontend origin in login/CSRF checks and keep `X-CSRF-Token` validation on authenticated mutations.

An HTTPS frontend must not call an HTTP API. Any configured Content Security Policy must allow the direct backend origin in `connect-src`. Use a same-origin proxy when direct browser sessions cannot be made reliable in supported browsers.

Confirmation sequence

1. Open browser developer tools and inspect the OPTIONS preflight for login/mutation requests.
2. Confirm the allowed Origin is exact and Access-Control-Allow-Credentials is true.
3. Confirm the login response sets the expected cookie and a subsequent GET /workspace sends it.
4. Confirm authenticated POST/DELETE requests send X-CSRF-Token and the server validates it.
5. Verify both an allowed frontend origin and a disallowed origin. Also check error responses carry the allowed CORS headers so the browser can display their safe messages.

Do not test only with curl. Curl does not enforce browser CORS, SameSite or third-party cookie restrictions. Browser end-to-end evidence is required for the selected topology.

IMPLEMENTATION

10 / Endpoint map and implementation order

Phase	Routes	Exit criterion
1. Identity	POST /auth/login; GET /auth/session; POST /auth/logout	Both roles authenticate; cookie/CSRF lifecycle and anonymous session work.
2. Authorized read model	GET /workspace; GET /leases/{id}	Tenant and organization isolation; correct IDs, dates, cents and summary.
3. Property / requests	POST /properties; POST /requests; POST /requests/{id}/reply	NC registration and cross-role clarification workflow work.
4. Policy lifecycle	POST /policies; DELETE /policies/{id}	Async ingestion, immutable versions, concurrency and cancellation behave correctly.
5. Assistant / review	POST /assistant/messages	Scoped retrieval and final answer checks fail closed; review metadata is fresh and bound.

Shared success rules

All 11 operations return HTTP 200 with a data envelope on success. Protected routes require a session; authenticated mutations require X-CSRF-Token. Every operation's complete schema is in the OpenAPI file and the dictionary at the end of this handbook.

Public versus protected

Only login and session are declared available before authentication. The public session route must safely return an anonymous Session. Property registration and policy writes require the landlord's policies:write permission under the current contract; do not invent a differently named permission without coordinating UI/schema changes.

Implemented browser interface

The API Explorer can test any documented route, including operations a tenant cannot use. The backend must enforce role and object checks regardless of the request source. The ordinary app exposes operations according to its workspace role and permissions.

ENDPOINT REFERENCE

E01 / POST /auth/login

`POST`

`/api/v1/auth/login`

Purpose: Authenticate; role comes from server identity

Access: Public credential exchange; server selects role and permissions.

Request

Content type: `application/json`. Model: `LoginInput`.

JSON example; replace IDs/credentials with your test records

```
{
  "email": "alex@alderliving.example",
  "password": "replace-with-test-credential"
}
```

Server implementation sequence

1. Validate credentials with generic failure messages, rate limiting and a trusted-origin login policy.
2. Rotate/create the session; set the session cookie and return User plus csrfToken.
3. Never accept role, tenantId or permissions from this login body. Those fields exist only in mock login.

Success: HTTP 200 with `{ "data": <Session> }`. See the complete `Session` field reference and companion examples.

Verification: Use real test accounts for both roles. Check HTTP 200, Set-Cookie in browser tools, and a schema-valid Session. Wrong credentials must not leak account existence.

Failure behavior: 401 invalid credentials; 422 invalid fields; 429 rate limit. Response details must remain user-safe.

ENDPOINT REFERENCE

E02 / GET /auth/session

`GET`

`/api/v1/auth/session`

Purpose: Anonymous session returns user:null and csrfToken:null

Access: Public session lookup; no CSRF header required for GET.

Request

No request body is required.

Server implementation sequence

1. Read and validate the server session cookie if present.
2. Return the authenticated User and CSRF token when valid.
3. Return user:null and csrfToken:null for an anonymous visitor; do not treat ordinary anonymity as 401.

Success: HTTP 200 with `{ "data": <Session> }`. See the complete `Session` field reference and companion examples.

Verification: Test before login, after login, after reload and after logout. The app restores its workspace from this route.

Failure behavior: A missing cookie is a successful anonymous Session. Use 503 for genuine service unavailability, not a fabricated authenticated user.

ENDPOINT REFERENCE

E03 / POST /auth/logout

`POST`

`/api/v1/auth/logout`

Purpose: Invalidate session and clear cookie

Access: Authenticated user; X-CSRF-Token required.

Path and header parameters

Name	Location	Required	Type
X-CSRF-Token	header	Yes	string

Request

No request body is required.

Server implementation sequence

1. Validate the session and CSRF token.
2. Invalidate the server session and expire its browser cookie.
3. Return data:{signedOut:true}. The current application sends an empty JSON object; the explorer can send no body.

Success: HTTP 200 with `{ "data": <Logout> }`. See the complete `Logout` field reference and companion examples.

Verification: After a successful logout, GET /auth/session must be anonymous and the old session must not authorize protected reads.

Failure behavior: 401 expired session; 403 bad CSRF. Do not return 204, HTML or a bare boolean.

ENDPOINT REFERENCE

E04 / GET /workspace

`GET`

`/api/v1/workspace`

Purpose: Return authorized workspace projection only

Access: Authenticated user; tenant projection or landlord organization projection.

Request

No request body is required.

Server implementation sequence

1. Resolve the session to its tenant/organization scope before any data query.
2. Return user, asOf, timeZone, properties, leases, tenants, requests, policies and summary. Include state on property/policy records.
3. Compute summary from the returned authorized active set; expose ingestion and compliance as separate statuses.

Success: HTTP 200 with `{ "data": <Workspace> }`. See the complete `Workspace` field reference and companion examples.

Verification: Compare two tenants and two organizations. A tenant must never receive another tenant's records, including in summary or sources.

Failure behavior: 401 anonymous; 503 unavailable read model. Do not return empty mock data when an API call fails.

ENDPOINT REFERENCE

E05 / GET /leases/{id}

`GET`

`/api/v1/leases/{id}`

Purpose: Return owned or authorized lease with citations

Access: Authenticated landlord in owning organization, or the tenant authorized for this lease.

Path and header parameters

Name	Location	Required	Type
id	path	Yes	string

Request

No request body is required.

Server implementation sequence

1. Resolve the encoded path id as an opaque value.
2. Check organization/tenant ownership before returning the lease or any clauses.
3. Return LeaseDetail: lease, clauses, property and tenant. Source quotes must retain their real document references.

Success: HTTP 200 with `{ "data": <LeaseDetail> }`. See the complete `LeaseDetail` field reference and companion examples.

Verification: Use a real lease ID obtained from the current workspace. A foreign tenant lease must return 404 without identifying its owner.

Failure behavior: 404 invisible/missing lease; 401 anonymous. Never infer authorization from an ID prefix.

ENDPOINT REFERENCE

E06 / POST /assistant/messages

`POST`

`/api/v1/assistant/messages`

Purpose: Retrieve authorized policy versions for explicit reference date

Access: Authenticated user; portfolio only for authorized landlords; lease scope must be owned/visible.

Path and header parameters

Name	Location	Required	Type
X-CSRF-Token	header	Yes	string

Request

Content type: `application/json`. Model: `AskInput`.

JSON example; replace IDs/credentials with your test records

```
{
  "question": "What does my lease say about repair requests?",
  "scope": {
    "type": "lease",
    "id": "lease-1"
  },
  "asOf": "2026-10-31",
  "threadId": null
}
```

Server implementation sequence

1. Validate the question, scope and real calendar asOf. Bind thread ownership/scope/date server-side.
2. Derive jurisdiction from authorized properties. Resolve policy versions and readiness before every retrieval branch.
3. Generate only supported content, validate the final answer and calculations, then return Answer with review metadata.
4. If evidence/review/coverage is incomplete, return a safe needs_clarification response with no unreviewed legal claims.

Success: HTTP 200 with `{ "data": <Answer> }`. See the complete `Answer` field reference and companion examples.

Verification: Test lease and portfolio scopes; foreign leases; changing reference dates; pending replacements; expired reviews; altered final text; missing official citations.

ENDPOINT REFERENCE

E06 / POST /assistant/messages / continued

Failure behavior: 403 unauthorized portfolio; 404 invisible lease/thread; 422 invalid input. Safe clarification is a successful 200 Answer, not a guessed answer.

ENDPOINT REFERENCE

E07 / POST /requests

`POST`

`/api/v1/requests`

Purpose: Tenant creates an owned-lease clarification request

Access: Authenticated tenant; leaseId must belong to that tenant.

Path and header parameters

Name	Location	Required	Type
X-CSRF-Token	header	Yes	string

Request

Content type: `application/json`. Model: `RequestInput`.

JSON example; replace IDs/credentials with your test records

```
{
  "leaseId": "lease-1",
  "category": "Maintenance",
  "subject": "Repair process",
  "message": "Please clarify the repair notification process."
}
```

Server implementation sequence

1. Validate subject/message lengths and the chosen visible lease.
2. Assign tenantId from the session; create an open request with a server timestamp.
3. Return the complete Request, including reply:null.

Success: HTTP 200 with `{ "data": <Request> }`. See the complete `Request` field reference and companion examples.

Verification: The request appears only for its tenant and owning landlord organization. Another tenant cannot read or reply to it.

Failure behavior: 403 landlord creating a tenant-only request; 404 invisible lease; 422 invalid content. Request category is a string, not the policy-category enum.

ENDPOINT REFERENCE

E08 / POST /requests/{id}/reply

`POST`

`/api/v1/requests/{id}/reply`

Purpose: Landlord replies to an authorized request

Access: Authenticated landlord authorized for the request's organization.

Path and header parameters

Name	Location	Required	Type
id	path	Yes	string
X-CSRF-Token	header	Yes	string

Request

Content type: `application/json`. Model: `ReplyInput`.

JSON example; replace IDs/credentials with your test records

```
{
  "reply": "Please send the details so we can review your request."
}
```

Server implementation sequence

1. Resolve the request under the current organization and validate reply text.
2. Record the reply, mark status answered and retain audit identity/time internally.
3. Return the complete updated Request, not a reply-only object.

Success: HTTP 200 with `{ "data": <Request> }`. See the complete `Request` field reference and companion examples.

Verification: The owning tenant sees the reply after workspace refresh. Foreign organizations and tenants cannot reply.

Failure behavior: 403 wrong role; 404 invisible request; 422 blank/invalid reply. No separate request-list endpoint is defined; workspace carries requests.

ENDPOINT REFERENCE

E09 / POST /policies

`POST`

`/api/v1/policies`

Purpose: Atomically append immutable version and close predecessor interval

Access: Authenticated landlord with policies:write; PDF multipart upload.

Path and header parameters

Name	Location	Required	Type
X-CSRF-Token	header	Yes	string

Request

Content type: `multipart/form-data`. Model: `PolicyUpload`.

Multipart field map for illustration; choose a real file in the explorer

```
{
  "file": "<binary native-text PDF part; not this placeholder string>",
  "title": "Maintenance policy",
  "category": "maintenance",
  "propertyId": "property-1",
  "state": "NC",
  "effectiveFrom": "2026-11-02"
}
```

Server implementation sequence

1. Validate private PDF bytes, nonempty size < =10 MiB, type/magic, native text availability and safe storage handling.
2. Validate state NC against every scoped property. New families require title/category; versions require familyId/expectedVersion.
3. In one transaction, lock the family, enforce chronological effectiveFrom, append the version and close its predecessor interval.
4. Return PolicyVersion promptly, with ingestion uploaded/processing and compliance pending. Process/review asynchronously.

Success: HTTP 200 with `{ "data": <PolicyVersion> }`. See the complete `PolicyVersion` field reference and companion examples.

Verification: Upload one PDF per request. Test new families, a new version, stale expectedVersion, invalid state/file, processing failure and effective-date blocking.

ENDPOINT REFERENCE

E09 / POST /policies / continued

Failure behavior: 409 VERSION_CONFLICT for stale/concurrent/date conflicts; 413 too large; 422 invalid PDF/fields. Detailed lifecycle rules follow in chapter 11.

ENDPOINT REFERENCE

E10 / DELETE /policies/{id}

DELETE

`/api/v1/policies/{id}`

Purpose: Cancel only latest future non-ready version; restore predecessor boundary

Access: Authenticated landlord with policies:write; owning organization only.

Path and header parameters

Name	Location	Required	Type
id	path	Yes	string
X-CSRF-Token	header	Yes	string

Request

No request body is required.

Server implementation sequence

1. Treat DELETE as cancellation of an eligible staged future version, not permanent history deletion.
2. Require latest version, future effectiveFrom, non-ready ingestion and no successor.
3. Atomically restore the predecessor's prior open boundary, remove it from active workspace output and retain a cancellation audit record.
4. Return data:{deleted:true}.

Success: HTTP 200 with `{ "data": <Deleted> }`. See the complete `Deleted` field reference and companion examples.

Verification: Cancel a future pending version and verify its predecessor remains applicable. Reject attempts to delete an effective, ready or superseded revision.

Failure behavior: 409 VERSION_CONFLICT for immutable/ineligible history; 404 invisible/missing version. Use new versions for corrections.

ENDPOINT REFERENCE

E11 / POST /properties

`POST`

`/api/v1/properties`

Purpose: Register a property with its required state; landlord only

Access: Authenticated landlord with policies:write; organization-scoped creation.

Path and header parameters

Name	Location	Required	Type
X-CSRF-Token	header	Yes	string

Request

Content type: `application/json`. Model: `PropertyRegistration`.

JSON example; replace IDs/credentials with your test records

```
{
  "name": "Oak House",
  "address": "10 Oak Street",
  "city": "Raleigh, NC 27601",
  "state": "NC",
  "units": 2
}
```

Server implementation sequence

1. Validate trimmed nonempty name/address/city; NC state; integer units from 1 through 10000.
2. Create the property under the authenticated organization, not a browser-supplied owner.
3. Return a complete Property including opaque id, state, style and amenities. Empty amenities is valid.

Success: HTTP 200 with `{ "data": <Property> }`. See the complete `Property` field reference and companion examples.

Verification: NC registration succeeds. Missing/foreign state, invalid units, tenant writes and cross-organization assignment fail.

Failure behavior: 403 wrong role/permission; 422 invalid values. HTTP 200 is the declared creation success status.

IMPLEMENTATION

11 / Policy storage, versions and async processing

A family is one policy's identity across versions. PolicyVersion.id is a unique immutable revision ID, familyId is stable, version is monotonically increasing. Title, category and property scope remain fixed within the family. A propertyId of null means organization-wide. Multiple families and categories may apply simultaneously; property-specific policies do not silently override global policies. Explicit precedence/conflict rules need a future contract if desired.

A version applies when effectiveFrom <= asOf AND (effectiveTo IS NULL OR asOf < effectiveTo). End dates are exclusive. For example, v1 [2026-01-01,2026-11-15) and v2 [2026-11-15,null) have exactly one applicable version on November 15: v2. Show the old end as 'until before November 15' to avoid ambiguity. Interpret calendar dates in the policy's immutable property time zone or the organization time zone for global policies. Persist that time zone with the policy family even though this frontend's Workspace supplies the display context.

Uploading a new version takes file, state, familyId, expectedVersion and effectiveFrom. New families take file, state, title, category, optional propertyId and effectiveFrom. Existing families inherit title/category/scope. Validate a real calendar date. Only append chronological versions: no earlier than today's server business date, and strictly later than the latest version's start. Retroactive corrections are not supported by this UI.

Within one database transaction, lock the family, verify expectedVersion matches the latest revision (otherwise 409 VERSION_CONFLICT), append vN+1, and set vN.effectiveTo to the new start. Use a uniqueness constraint on (familyId,version), enforce non-overlap and protect concurrent writes. Old bytes, checksums, extraction outputs, uploader identity and citations remain immutable for audit. Separate uploaded timestamp from policy effective date.

A future version is scheduled; it does not replace the current policy early. At its start, the predecessor is ineligible even if the new file has not finished processing. Warn administrators before activating an unready policy. Return a clear 'policy ingestion unavailable' clarification result; never silently retrieve the expired version. Ingestion failure must remain visible and require retry or a later corrected version, not a rollback of history.

DELETE is narrowly a cancellation operation: only the latest future non-ready version with no successor may be canceled. Restore its predecessor's previous open-ended boundary atomically. Effective/current versions and ready historical revisions cannot be deleted. Keep a separate cancellation audit record in the backend even if the workspace hides the canceled revision. The UI stages `awaiting_backend` only in mock mode; Python uses uploaded → processing → ready or failed. Ready means validated extraction and all required indices are committed. No partial-ready retrieval.

Validate file size (max 10 MiB), PDF magic/type, native text availability, malware policy and safe filename/storage handling. The browser extension filter is only a convenience. Store uploaded bytes privately, never in public frontend assets. Return metadata promptly and process asynchronously. The current UI uses Refresh to reload ingestion status; polling, job progress and retries are future extensions. No actual file contents are stored in mock mode.

IMPLEMENTATION

11 / Policy storage, versions and async processing /

continued

Ingestion and legal review are independent

A ready document is fully extracted and indexed, not automatically law-compliant. The backend must require both ready ingestion and a current passed legal review before retrieval. Do not overload status with legal outcomes; use compliance.status for those.

Job integration without extra UI endpoints

The HTTP response represents a durable accepted version, not a finished job. Workers update stored ingestion/compliance metadata. The current UI reloads it through GET /workspace when Refresh is chosen. No job-status, retry, progress stream or ingestion callback endpoint is defined for the frontend yet. Implement internal queues/callbacks privately and coordinate any browser-visible extension.

Dates and concurrency: worked boundary

If v2 is effective from 2026-07-01 and v3 starts 2026-11-15, v2.effectiveTo becomes 2026-11-15. Querying November 14 selects v2; querying November 15 selects v3. If v3 is still processing or its review is unavailable, the answer must clarify. It must not use v2 as a substitute. A second writer using expectedVersion=2 after v3 exists must receive 409.

Upload response and retries

Persist the accepted upload and version metadata transactionally with a recoverable job/outbox mechanism appropriate to your stack. No Idempotency-Key header is specified. If a client times out, reload workspace before retrying; do not assume timeout means nothing was saved. A staged upload may be accepted even if a worker or legal source is temporarily unavailable, but must remain visibly ineligible for guidance.

IMPLEMENTATION

12 / NC state and compliance behavior

Jurisdiction is property data

Property registration captures state: NC. Every corpus/version upload is tagged to NC and must match all properties in its scope. New versions inherit the family's state, category and property scope. Separate states would require separate policy families and coordinated source/UI/schema support; only NC is configured now.

Tenants never choose a state. Resolve it from the authorized lease and property. Portfolio legal questions must be evaluated per authorized property; the current single-state wire guard expects NC. A policy with propertyId:null is organization-wide within its state; do not accidentally apply it to a later property in another state.

Shared laws, separate assessments

Fetch government-source snapshots once per state and reuse them for uploads and answers. The reference cache refreshes after one day and stores source content, URLs, fetchedAt, expiresAt and a content-derived revision. Reusing the laws does not reuse a verdict for a different document or answer.

The configured sources are the NC General Assembly Chapter 42 HTML and the NC Department of Justice renting guidance. The reference fetcher rejects redirects and oversized/unexpected source responses. A missing source, refresh failure or unsupported state blocks review. Add source monitoring/recovery so a changed government URL is not mistaken for a valid empty corpus.

Reviewer outcomes

- pending: document/source data may exist, but review has not established complete coverage.
- passed: trusted review of the exact content, scope and date completed with current source metadata, official citations and no unresolved findings.
- conflict: rules or claims require changes; preserve useful clause-level findings for review.
- unavailable: the required source/reviewer/check could not be completed or verified.

No model confidence score substitutes for coverage. Uploaded text is untrusted evidence, never reviewer instructions. Assess applicable exceptions, amendments and federal/local requirements too. A cached current chapter cannot establish historical/future law by itself. Do not approve an unsupported date merely because a current source was downloaded.

Default-deny behavior

The Python module returns pending when no reviewer is connected. It is a reusable boundary, not a legal reasoning engine or HTTP server. The backend must supply qualified review logic and enforce the result before releasing guidance. The frontend's display checks are additional safeguards, not a security or legal certification boundary.

IMPLEMENTATION

13 / Exact review metadata and hashing

Required metadata when status is passed

The review identifies state, kind, scopeKey, exact asOf, checkedAt, expiresAt, 64-character lowercase hexadecimal lawRevision/contentHash and at least one official citation with a nonempty section. Findings must be absent or empty on a passed result. The metadata object rejects undeclared top-level fields.

Review target	kind	Exact scopeKey
Lease answer	answer	lease:<authorized lease id>
Portfolio answer	answer	portfolio:all
Property-specific policy	policy	property:<authorized property id>
Organization-wide NC policy	policy	state:NC

Answer identity

Hash the exact final UTF-8 answer string, not its JSON serialization, citations, a normalized copy, or an earlier draft. Preserve whitespace and newlines consistently. The backend validates the final answer and all calculations before release; its internal audit record must bind calculation results too, even though this wire contentHash checks the answer string only.

Python: final answer hash

```
import hashlib
content_hash = hashlib.sha256(
    final_answer.encode("utf-8")
).hexdigest()
```

Policy identity

Define and persist the canonical extracted text used for review; hash exactly those UTF-8 bytes. Return that value in both PolicyVersion.contentHash and compliance.contentHash. Preserve the original PDF checksum separately in storage/audit. Do not hash a short excerpt or mutate the canonical text after approval.

What the current frontend actually checks

For supported API answers, it checks NC, exact asOf/kind/scopeKey, parseable timestamps with checkedAt <= now < expiresAt, hash formats, allowed citation URLs/sections, empty findings and the exact answer hash. The policy helper additionally compares its stored contentHash and property/state scope. It checks the shape of lawRevision; it cannot independently know the backend's latest law revision or your reviewer quality. The backend must enforce active law/source validity, organization ownership and revocation.

All supported API answers, including portfolio summaries, pass through the answer-review guard. The mock portfolio's example behavior is not an exemption in API mode. Any genuinely exempt factual-response category needs a coordinated contract/UI change.

IMPLEMENTATION

14 / Reviewer integration and safe clarification

Reference Python call

The module uses only Python's standard library. Point LawCache at a writable persistent SQLite file; the caller supplies trusted scope and property states after authorization. Its BEGIN IMMEDIATE lock deduplicates a refresh across processes sharing that file. At production scale, choose the storage/locking strategy appropriate to your deployment rather than placing unrelated work behind a global file lock.

Reference boundary; application/framework plumbing omitted

```
from laga.compliance import LawCache, review

cache = LawCache("/private-data/state-laws.sqlite")
assessment = review(
    cache,
    state="NC",
    property_states=["NC"],
    text=final_answer,
    as_of=request.as_of,
    kind="answer",
    scope_key="lease:" + authorized_lease.id,
    reviewer=trusted_server_reviewer,
)
# Authorize + validate calculations separately.
# Release supported guidance only if assessment passed.
```

The reviewer receives text, laws, as_of, kind and scope_key as keyword arguments. A passing assessment must include status:passed, coverageComplete:true, official citations and no findings. These are outputs of trusted server code, never fields the browser is allowed to assert. The wrapper checks expiration again after review; a review that outlives its source snapshot is unavailable.

Safe clarification response

When an applicable policy is pending, evidence conflicts or legal coverage is insufficient, return HTTP 200 Answer with status:needs_clarification and an explanation of the missing check. Do not include the unreviewed legal advice anyway and hope the status will hide it: the frontend renders clarification text as well. Omit unsafe calculations and unrelated citations. Set compliance to pending/conflict/unavailable as appropriate.

Illustrative complete clarification envelope

```
{
  "data": {
    "id": "answer-example",
    "threadId": "thread-example",
    "answer": "The applicable policy is awaiting legal review. Please request a review before relying on its
terms.",
    "status": "needs_clarification",
    "citations": [],
    "calculations": [],
    "activity": [
```

IMPLEMENTATION

14 / Reviewer integration and safe clarification /

continued

```
      {
        "label": "Legal review is pending",
        "status": "attention"
      }
    ],
    "compliance": {
      "status": "pending",
      "state": "NC"
    }
  }
}
```

IMPLEMENTATION

15 / Manual HTTP walkthrough: curl

These examples are for a development/test backend, not credentials to paste into production. Replace IDs and the base URL with records from your own authorized workspace. On Windows PowerShell, use curl.exe so the command is not treated as an Invoke-WebRequest alias. For cross-origin testing, the Origin below must be allowed by your server.

Step 1: prepare request files

Create login.json with a real test account and property.json with the registration fields shown in E11. Do not commit real credentials. The OpenAPI and integration examples provide the exact shapes. Keep private test files out of shared archives.

Step 2: establish a cookie session

POSIX shell; inspect the returned HTTP/body before proceeding

```
BASE="http://localhost:8001/api/v1"
UI="http://localhost:5174"
curl --silent --show-error --include \
  -c cookies.txt -b cookies.txt \
  "$BASE/auth/session"

curl --silent --show-error \
  -c cookies.txt -b cookies.txt \
  -H "Origin: $UI" \
  -H "Content-Type: application/json" \
  --data-binary @login.json \
  "$BASE/auth/login" > login-response.json
```

Step 3: use the returned CSRF token

Keep the token in your test environment; do not publish it

```
CSRF=$(node -p \
  "JSON.parse(require('fs').readFileSync('login-response.json','utf8')).data.csrfToken")

curl --silent --show-error --include \
  -b cookies.txt "$BASE/workspace"

curl --silent --show-error --include \
  -b cookies.txt -H "Origin: $UI" \
  -H "X-CSRF-Token: $CSRF" \
  -H "Content-Type: application/json" \
  --data-binary @property.json "$BASE/properties"
```

IMPLEMENTATION

15 / Manual HTTP walkthrough: curl / continued

Step 4: upload a real policy

Use a valid effective date at execution time; do not set the multipart Content-Type header yourself

```
curl --silent --show-error --include \
  -b cookies.txt -H "Origin: $UI" \
  -H "X-CSRF-Token: $CSRF" \
  -F "file=@rules.pdf;type=application/pdf" \
  -F "title=Maintenance policy" \
  -F "category=maintenance" \
  -F "propertyId=REPLACE_WITH_REAL_PROPERTY_ID" \
  -F "state=NC" -F "effectiveFrom=2026-11-15" \
  "$BASE/policies"
```

For a new version, send file, familyId, expectedVersion, state and effectiveFrom instead of changing the family's title/category/scope. Re-read the current workspace before choosing expectedVersion. Inspect status and JSON on every call; a shell command completing is not proof of HTTP success. Curl cannot verify browser cookie/CORS behavior.

IMPLEMENTATION

16 / Manual HTTP walkthrough: PowerShell

JSON calls with a shared WebRequestSession

This uses standard PowerShell JSON request/session handling. Supply your backend address and credentials in a private login.json file; the response retains the browser-equivalent session cookies in the web variable.

PowerShell; HTTP errors must be inspected rather than ignored

```
$base = "http://localhost:8001/api/v1"
$ui = "http://localhost:5174"
$loginBody = Get-Content -LiteralPath login.json -Raw

$login = Invoke-RestMethod -Method Post `
  -Uri "$base/auth/login" `
  -ContentType "application/json" `
  -Headers @{ Origin = $ui } `
  -Body $loginBody -SessionVariable web

$csrf = $login.data.csrfToken
$headers = @{
  Origin = $ui
  "X-CSRF-Token" = $csrf
}

$workspace = Invoke-RestMethod -Method Get `
  -Uri "$base/workspace" -WebSession $web

$propertyBody = @{
  name = "Oak House"
  address = "10 Oak Street"
  city = "Raleigh, NC 27601"
  state = "NC"
  units = 2
} | ConvertTo-Json

$created = Invoke-RestMethod -Method Post `
  -Uri "$base/properties" -WebSession $web `
  -Headers $headers -ContentType "application/json" `
  -Body $propertyBody
```

Multipart calls

PowerShell 7 supports Invoke-RestMethod -Form with a Get-Item file value. Windows PowerShell 5.1 does not provide the same -Form interface; use the API Explorer or curl.exe for multipart uploads. Do not mix a PowerShell WebRequestSession cookie store with an unrelated curl cookie jar and assume they share authentication. Use one client/session end-to-end.

IMPLEMENTATION

16 / Manual HTTP walkthrough: PowerShell / continued

Get real identifiers from the read model

Use $workspace.data.properties, leases, requests and policies. A policy's id identifies one immutable revision; familyId identifies its version series. Never substitute a familyId into a DELETE revision path, and never treat a tenant user ID as a lease ID.

IMPLEMENTATION

17 / Acceptance matrix and test evidence

Area	Required positive test	Required negative/boundary test
Session	Both roles log in; refresh restores session; logout invalidates it.	Anonymous session is 200/null; expired protected access is 401.
Isolation	Tenant sees only their lease/property; landlord sees own organization.	Foreign lease/thread/request/document IDs disclose no other account data.
Registration	NC property creates a complete Property DTO.	Tenant write, missing state, TX state, zero/fractional units fail.
CORS / cookies	Browser login + credentialed GET/POST work for allowed origin.	Disallowed origin, bad CSRF and missing cookie fail safely.
Policy upload	Native-text PDF returns durable pending metadata.	Empty/non-PDF/oversized file, bad state and unavailable property fail.
Versioning	Replacement updates predecessor boundary atomically.	Two writers with same expectedVersion: one success, one 409.
Date filter	Before/at effective boundary selects exactly one version.	Pending/failed replacement does not resurrect expired content.
Cancellation	Latest staged future pending revision can be canceled.	Effective, ready, historical and superseded versions are immutable.
Legal review	Exact document/answer/scope/date receives current trusted assessment.	Wrong hash/date/state/scope, expired review, unknown citations or findings block use.
Assistant	Citations and cent calculations match authorized evidence.	Unclear/conflicting/unready evidence returns safe clarification.
Requests	Tenant creates; owning landlord replies; tenant sees update.	Other tenants/organizations cannot read or reply.
Failure handling	Safe JSON errors preserve requestId and field details.	HTML/204/raw DTO/API outage do not silently activate mock mode.

Record evidence, not just checkmarks

For each environment, retain contract version, backend build identifier, applied API base, account roles, scenario/date, status/result and relevant server request IDs. Use non-production test documents and redact tenant/credential data. The explorer report deliberately excludes bodies, passwords and tokens; it is structural evidence, not a complete security/legal test report.

Run npm test and npm run build in the frontend repository. Run python -m unittest discover -s tests -v for the reusable cache/review boundary. Add backend endpoint tests and run browser/application acceptance against your actual service; existing fixtures are not tests of your deployed backend.

IMPLEMENTATION

18 / Production cutover and rollback

Pre-cutover checklist

1. Freeze the accepted OpenAPI contract and record its version/hash with the backend release.
2. Provision production roles/organizations/properties and verify NC state associations without demo records.
3. Confirm private file storage, extraction/indexing jobs, legal-source freshness monitoring and reviewer availability.
4. Configure TLS, trusted proxy headers, cookie flags, CSRF/origin checks and CORS/CSP where required.
5. Set the application build environment to api mode and the chosen base, build, deploy and inspect the generated asset release.
6. Validate session → workspace → lease → request/reply → policy lifecycle → assistant flows using both roles on the actual deployed topology.
7. Restrict developer tooling to the intended testing audience or remove its entry point according to the release plan. Hiding a toggle alone does not protect /api-docs/.

No automatic promotion

Applying a server in the explorer does not update the application. A successful test report does not deploy it. VITE_* values are baked into the built frontend, so changes require rebuilding/redeploying. LAGA_API_PROXY only affects the development server.

Release/rollback behavior

Keep a known-good frontend asset release and backend release identifier. Roll back compatible deployments deliberately; do not alter effective policy history to compensate for a failed release. If a legal reviewer/source is unavailable, keep affected guidance blocked rather than switching users to mock data or a previous expired policy.

Operational ownership

Assign owners for session failures, API availability, ingestion failures, source refresh failures, review conflicts and retrieval integrity. Attach request IDs to safe logs; do not log passwords, CSRF tokens, raw tenant documents or private model reasoning. Define your retention, deletion, incident response and legal review service levels before production; this API version does not prescribe numerical SLAs.

IMPLEMENTATION

19 / Troubleshooting by symptom

Symptom	Check
`/api` returns 503 `BACKEND_NOT_CONNECTED`	Static server is receiving the request; add a reverse proxy or use a full API URL
404 on all API endpoints	Full base prefix should be `/api/v1`; proxy must preserve it exactly once
HTML or login page instead of JSON	SPA fallback or authentication gateway intercepted `/api`; fix routing
“Cannot reach this API”	Server/TLS, browser console, HTTPS mixed content, CORS or CSP
Login succeeds but workspace is 401	Cookie host/path, credentials, SameSite/Secure, browser cookie policy
Mutations are 403	Capture session CSRF token; verify origin allowlist and token header validation
“Response does not match contract”	Compare response envelope and DTO with the model inspector/OpenAPI
Answer withheld after a passed review	Review expiry, exact final-text hash, NC state, scope, reference date and official citations
App still displays demo mode	Set `VITE_LAGA_MODE=api`, restart Vite or rebuild/redeploy

Additional integration checks

Symptom	Likely cause / correction
A create route is successful but explorer flags status	Backend returned 201; this contract specifies 200. Align the backend or coordinate a contract/UI change.
Login works in curl, not browser	Curl does not enforce CORS, SameSite, mixed-content or third-party cookie policy. Inspect browser requests.
New version fails enum validation	Omit inapplicable multipart fields; do not send literal undefined/null strings. Existing family scope/category/state remain fixed.
Policy is ready but not eligible	Ingestion readiness and legal review are separate; inspect current compliance and exact contentHash/scope/date.
Review passes for yesterday but not today	The wire guard requires exact asOf. Refresh/reassess using date-appropriate source coverage. Do not fabricate a new date on an old verdict.
Review expires while model/reviewer is running	Reference boundary blocks it. Refresh the snapshot and rerun the assessment, subject to rate/retry policy.
Official source URL redirects	Reference fetcher rejects redirects. Verify and update the trusted source configuration; do not approve an HTML redirect/challenge page.

IMPLEMENTATION

19 / Troubleshooting by symptom / continued

Symptom	Likely cause / correction
Developer Tools is absent on a lease page	Intentional: use login or home dashboard; the direct /api-docs/ route remains available.
An example review immediately fails	Fixed fictional timestamps may be expired. Generate a fresh genuine review; do not use fixture approvals in production.
Upload canceled but a version appears	Browser cancel does not roll back server work. Refresh and reconcile the actual accepted version before retrying.

IMPLEMENTATION

20 / Decisions to close before implementation

The following details are not invented by this handbook. The integration owner should record the chosen implementation alongside the release so the frontend and backend teams have one agreement.

Decision	Owner / required outcome
Backend runtime and deployment	Backend + hosting: framework, base URL, environments, reachable upstream and health/monitoring approach.
Tenant/landlord account provisioning	Backend/product: identity provider, registration/password recovery and lease association workflow. No signup/provisioning endpoints are in this API version.
Unassigned tenant behavior	Product + frontend/backend: provision the lease association and verify the existing home empty state plus any other lease-dependent controls.
Canonical policy text	Ingestion owner: exact extracted-text representation/UTF-8 bytes and hash persistence; preserve original PDF checksum too.
Legal reviewer and coverage	Legal/compliance + backend: qualified reviewer, applicable exceptions/amendments/federal/local coverage and unsupported-date policy.
Findings object shape	Current schema permits object findings; UI renders message or description. Standardize clause/section details internally; extra presentation fields require coordination.
Retry/idempotency strategy	Backend: safe mutation recovery after timeouts. No Idempotency-Key or public job/retry endpoint is specified.
Retention and access audit	Security/operations: immutable version and cancellation audit, private storage retention and incident ownership.
Future contract extensions	Joint owner: pagination, streaming, job progress, document downloads, multi-state support or a new factual-answer exemption require coordinated schemas/adapter work.

Compatibility rule

Do not replace an established response shape with your database model or add undeclared top-level fields. Additive optional fields, new routes or changed status codes still need contract and adapter review. Keep the spec, behavioral guide, examples, explorer and backend tests synchronized.

What is not promised

There is no automated production connection, no included production database/RAG backend, no guarantee that a source snapshot is a complete statement of applicable law, and no legal certification from a successful HTTP/schema check. These are implementation and acceptance responsibilities, not missing URL settings.

COMPLETE DATA DICTIONARY

A1 / Identity and session models

Generated from the current OpenAPI schemas. Required means the key must be present; nullable means null is permitted as a value. Semantic/backend checks still apply.

User

Additional undeclared properties: **rejected**.

User fields

Field	Type / required	Meaning / constraints
`id`	string · Required	Opaque stable identifier. Authorize the underlying object independently of its format.
`name`	string · Required	Human-readable name; never use as the authorization key.
`email`	string · Required	Account email returned by the server; login accepts email/password only.
`role`	string · Required	Wire role. The UI label Customer corresponds to tenant. Allowed: tenant, landlord.
`organization`	string · Optional	Display organization name, not proof of membership.
`permissions`	array<string> · Required	Server-issued permission strings used for UI visibility; server checks remain mandatory.
`unit`	string · Optional	Display unit identifier; a string, not necessarily a number.
`propertyId`	string · Optional	Authorized property association. A nullable policy scope of null means organization-wide within its state.
`leaseId`	string · Optional	Authorized lease association; never accept a foreign tenant lease.

Session

Additional undeclared properties: **rejected**.

Session fields

Field	Type / required	Meaning / constraints
`user`	object · Required	Authenticated user, or null in an anonymous Session.

COMPLETE DATA DICTIONARY

A1 / Identity and session models / continued

Session fields

Field	Type / required	Meaning / constraints
`csrfToken`	string	null · Required	Opaque CSRF token returned with session data; held in page memory, sent on authenticated mutations.

COMPLETE DATA DICTIONARY

A2 / Properties and workspace

Generated from the current OpenAPI schemas. Required means the key must be present; nullable means null is permitted as a value. Semantic/backend checks still apply.

PropertyRegistration

Additional undeclared properties: **rejected**.

PropertyRegistration fields

Field	Type / required	Meaning / constraints
`name`	string · Required	Human-readable name; never use as the authorization key. Min length 1; Max length 120.
`address`	string · Required	Street address display field. Min length 1; Max length 250.
`city`	string · Required	City/ZIP display string in the existing UI; not a separate state source. Min length 1; Max length 120.
`state`	string · Required	Jurisdiction code. NC is the only configured state in this contract. Allowed: NC.
`units`	integer · Required	Integer number of property units. Min 1; Max 10000.

Property

Additional undeclared properties: **rejected**.

Property fields

Field	Type / required	Meaning / constraints
`id`	string · Required	Opaque stable identifier. Authorize the underlying object independently of its format.
`name`	string · Required	Human-readable name; never use as the authorization key.
`address`	string · Required	Street address display field.
`city`	string · Required	City/ZIP display string in the existing UI; not a separate state source.
`units`	integer · Required	Integer number of property units.
`style`	string · Required	Required frontend illustration key; use a supported default. Allowed: willow, juniper, cedar.

COMPLETE DATA DICTIONARY

A2 / Properties and workspace / continued

Property fields

Field	Type / required	Meaning / constraints
`amenities`	array<string> · Required	Display amenity strings; an empty array is allowed.
`description`	string · Optional	Display description; also used as a fallback for a review finding.
`state`	string · Required	Jurisdiction code. NC is the only configured state in this contract. Allowed: NC.

Summary

Additional undeclared properties: **rejected**.

Summary fields

Field	Type / required	Meaning / constraints
`properties`	integer · Required	Authorized property records (or a property count in Summary).
`units`	integer · Required	Integer number of property units.
`occupied`	integer · Required	Occupied home count; avoid double counting shared leaseholders.
`monthlyRent`	integer · Required	Authorized active contracted monthly rent total in cents.
`outstandingRent`	integer · Required	Authorized outstanding ledger amount in cents.
`renewals`	integer · Required	Leases meeting the current renewal window in the read model.

Workspace

Additional undeclared properties: **rejected**.

Workspace fields

Field	Type / required	Meaning / constraints
`asOf`	string · Required	Requested/server business calendar date. Exact matching matters for review identity. Pattern: ^\d{4}-\d{2}-\d{2}$.

COMPLETE DATA DICTIONARY

A2 / Properties and workspace / continued

Workspace fields

Field	Type / required	Meaning / constraints
`timeZone`	string · Required	IANA time zone for date interpretation/display; configure real business rules, not fixture values.
`demo`	boolean · Optional	Optional preview marker. Real production behavior must not depend on demo fixtures.
`user`	User · Required	Authenticated user, or null in an anonymous Session.
`properties`	array<Property> · Required	Authorized property records (or a property count in Summary).
`leases`	array<Lease> · Required	Authorized lease records.
`tenants`	array<User> · Required	Authorized tenant User records only.
`requests`	array<Request> · Required	Authorized clarification requests.
`policies`	array<PolicyVersion> · Required	Policy version records, including readiness and compliance metadata.
`summary`	Summary · Required	Aggregates computed from the same authorized lease/property set.

COMPLETE DATA DICTIONARY

A3 / Leases, details and question scope

Generated from the current OpenAPI schemas. Required means the key must be present; nullable means null is permitted as a value. Semantic/backend checks still apply.

Lease

Additional undeclared properties: **rejected**.

Lease fields

Field	Type / required	Meaning / constraints
`id`	string · Required	Opaque stable identifier. Authorize the underlying object independently of its format.
`tenantId`	string · Required	Owning tenant identity; derive/validate against authenticated scope.
`tenantName`	string · Required	See the model and endpoint behavior for usage.
`propertyId`	string · Required	Authorized property association. A nullable policy scope of null means organization-wide within its state.
`unit`	string · Required	Display unit identifier; a string, not necessarily a number.
`rent`	integer · Required	Contracted monthly rent in integer USD cents.
`deposit`	integer · Required	Security deposit in integer USD cents.
`currency`	"USD" · Required	USD in the current contract. Constant: "USD".
`startDate`	string · Required	Lease start calendar date. Pattern: ^\d{4}-\d{2}-\d{2}$.
`endDate`	string · Required	Lease end calendar date; does not by itself determine statutory notice rights. Pattern: ^\d{4}-\d{2}-\d{2}$.
`status`	string · Required	Lifecycle/result state; see the model-specific enum and behavioral chapters. Allowed: active, ended, pending.
`paymentStatus`	string · Required	UI ledger state; do not invent amounts from this label. Allowed: current, due.
`dueDay`	integer · Required	Contractual rent-due day; validate legal applicability separately.

COMPLETE DATA DICTIONARY

A3 / Leases, details and question scope / continued

Lease fields

Field	Type / required	Meaning / constraints
`noticeDays`	integer · Required	Contractual notice period; not an independent legal ruling.
`renewalStatus`	string · Required	Backend renewal workflow display status.
`documents`	array<string> · Optional	Lease document IDs; no download route is specified yet.

LeaseDetail

Additional undeclared properties: **rejected**.

LeaseDetail fields

Field	Type / required	Meaning / constraints
`lease`	Lease · Required	The authorized structured lease.
`property`	Property · Required	Property associated with the selected lease.
`tenant`	User · Required	Tenant associated with the selected lease.
`clauses`	array<Citation> · Required	Authorized source Citation records for the selected lease.

Scope

Additional undeclared properties: **rejected**.

Scope fields

Field	Type / required	Meaning / constraints
`type`	string · Required	Scope discriminator: lease or portfolio. Allowed: lease, portfolio.
`id`	string | null · Required	Opaque stable identifier. Authorize the underlying object independently of its format.

> Server invariant: type lease requires a nonempty authorized lease id; portfolio uses id:null and landlord authorization. The base schema does not enforce this cross-field rule.

COMPLETE DATA DICTIONARY

A4 / Policy upload and immutable versions

Generated from the current OpenAPI schemas. Required means the key must be present; nullable means null is permitted as a value. Semantic/backend checks still apply.

PolicyUpload

Additional undeclared properties: **rejected**.

PolicyUpload fields

Field	Type / required	Meaning / constraints
`file`	string · Required	See the model and endpoint behavior for usage. Format: binary.
`familyId`	string · Optional	Stable identity across immutable policy versions; not a revision/document ID. Min length 1.
`expectedVersion`	integer · Optional	Optimistic concurrency token: latest known version before appending a revision. Min 1.
`title`	string · Optional	Document/source/policy title. Fixed across an existing policy family. Min length 1; Max length 120.
`category`	string · Optional	Policy enum for policies; free-form string for clarification requests. Allowed: tenant, pets, utilities, waste, community, maintenance, other.
`propertyId`	string | null · Optional	Authorized property association. A nullable policy scope of null means organization-wide within its state.
`effectiveFrom`	string · Required	Inclusive calendar date at which this version applies. Pattern: ^\d{4}-\d{2}-\d{2}$.
`state`	string · Required	Jurisdiction code. NC is the only configured state in this contract. Allowed: NC.

> Conditional requirement: familyId present → expectedVersion required; no familyId → title and category required. file, state and effectiveFrom are always required. Binary file data is a multipart part, not a JSON placeholder.

PolicyVersion

Additional undeclared properties: **rejected**.

COMPLETE DATA DICTIONARY

A4 / Policy upload and immutable versions / continued

PolicyVersion fields

Field	Type / required	Meaning / constraints
`id`	string · Required	Opaque stable identifier. Authorize the underlying object independently of its format.
`familyId`	string · Required	Stable identity across immutable policy versions; not a revision/document ID.
`title`	string · Required	Document/source/policy title. Fixed across an existing policy family.
`name`	string · Required	Human-readable name; never use as the authorization key.
`category`	string · Required	Policy enum for policies; free-form string for clarification requests. Allowed: tenant, pets, utilities, waste, community, maintenance, other.
`propertyId`	string | null · Required	Authorized property association. A nullable policy scope of null means organization-wide within its state.
`status`	string · Required	Lifecycle/result state; see the model-specific enum and behavioral chapters. Allowed: awaiting_backend, uploaded, processing, ready, failed.
`version`	integer · Required	Monotonically increasing positive integer within a policy family. Min 1.
`effectiveFrom`	string · Required	Inclusive calendar date at which this version applies. Pattern: ^\d{4}-\d{2}-\d{2}$.
`effectiveTo`	string | null · Required	Exclusive end boundary; null means open-ended. Pattern: ^\d{4}-\d{2}-\d{2}$.
`supersedesId`	string | null · Optional	Previous immutable revision ID when one exists.
`uploadedAt`	string · Required	Server timestamp with UTC offset; separate from effectiveFrom.
`size`	integer · Required	Uploaded file size in bytes.
`pageCount`	integer | null · Required	PDF page count; required but nullable while unknown.

COMPLETE DATA DICTIONARY

A4 / Policy upload and immutable versions / continued

PolicyVersion fields

Field	Type / required	Meaning / constraints
`uploadedBy`	string · Required	Uploader display name returned by the server; audit identity must be retained internally.
`description`	string · Required	Display description; also used as a fallback for a review finding.
`excerpt`	string | null · Required	Searchable/source excerpt; nullable while unavailable. Not a canonical full-document hash input.
`state`	string · Required	Jurisdiction code. NC is the only configured state in this contract. Allowed: NC.
`compliance`	Compliance · Required	See the model and endpoint behavior for usage.
`contentHash`	string · Optional	SHA-256 lowercase hex of exact reviewed content; match canonical policy text or final answer as specified. Pattern: ^[a-f0-9]{64}$.

COMPLETE DATA DICTIONARY

A5 / Assistant requests and evidence

Generated from the current OpenAPI schemas. Required means the key must be present; nullable means null is permitted as a value. Semantic/backend checks still apply.

AskInput

Additional undeclared properties: **rejected**.

AskInput fields

Field	Type / required	Meaning / constraints
`question`	string · Required	User question, 1–2000 characters; validate scope and evidence independently. Min length 1; Max length 2000.
`scope`	Scope · Required	Requested lease or portfolio context; server authorizes it.
`asOf`	string · Required	Requested/server business calendar date. Exact matching matters for review identity. Pattern: ^\d{4}-\d{2}-\d{2}$.
`threadId`	string | null · Optional	Conversation identity; optional/nullable on input, required on output. Never an access token.

Answer

Additional undeclared properties: **rejected**.

Answer fields

Field	Type / required	Meaning / constraints
`id`	string · Required	Opaque stable identifier. Authorize the underlying object independently of its format.
`threadId`	string · Required	Conversation identity; optional/nullable on input, required on output. Never an access token.
`answer`	string · Required	Final displayed text. If supported, the exact UTF-8 string must match its review hash.
`status`	string · Required	Lifecycle/result state; see the model-specific enum and behavioral chapters. Allowed: supported, needs_clarification.
`citations`	array<Citation> · Required	Evidence references. Answer citations and legal-review citations are different models.

COMPLETE DATA DICTIONARY

A5 / Assistant requests and evidence / continued

Answer fields

Field	Type / required	Meaning / constraints
`calculations`	array‹Calculation› · Required	Deterministic calculation records; server validates inputs/method/results before release.
`activity`	array‹Activity› · Required	User-safe check labels; never private model chain-of-thought.
`demo`	boolean · Optional	Optional preview marker. Real production behavior must not depend on demo fixtures.
`compliance`	Compliance · Required	See the model and endpoint behavior for usage.

Citation

Additional undeclared properties: **rejected**.

Citation fields

Field	Type / required	Meaning / constraints
`id`	string · Required	Opaque stable identifier. Authorize the underlying object independently of its format.
`title`	string · Required	Document/source/policy title. Fixed across an existing policy family.
`section`	string · Required	Source section identifier; nonempty for legal-review citations.
`page`	integer | null · Required	Source page number; null when page mapping is unavailable.
`quote`	string · Required	Exact authorized source excerpt used to substantiate an answer.
`documentId`	string · Required	Stable document/revision identity. Policy references must name the actual version.
`version`	integer · Optional	Monotonically increasing positive integer within a policy family.
`effectiveFrom`	string · Optional	Inclusive calendar date at which this version applies. Pattern: ^\d{4}-\d{2}-\d{2}$.

COMPLETE DATA DICTIONARY

A5 / Assistant requests and evidence / continued

Citation fields

Field	Type / required	Meaning / constraints
`effectiveTo`	string | null · Optional	Exclusive end boundary; null means open-ended. Pattern: ^\d{4}-\d{2}-\d{2}$.

Calculation

Additional undeclared properties: **rejected**.

Calculation fields

Field	Type / required	Meaning / constraints
`label`	string · Required	User-visible activity/calculation label.
`inputs`	object · Required	Named integer/string/boolean calculation inputs; monetary inputs use integer cents. Additional values: integer | string | boolean.
`result`	integer · Required	Integer calculation result in USD cents.
`currency`	"USD" · Required	USD in the current contract. Constant: "USD".
`method`	string · Required	Human-readable calculation method; never execute a client expression.

Activity

Additional undeclared properties: **rejected**.

Activity fields

Field	Type / required	Meaning / constraints
`label`	string · Required	User-visible activity/calculation label.
`status`	string · Required	Lifecycle/result state; see the model-specific enum and behavioral chapters. Allowed: complete, attention.

COMPLETE DATA DICTIONARY

A6 / Clarification request workflow

Generated from the current OpenAPI schemas. Required means the key must be present; nullable means null is permitted as a value. Semantic/backend checks still apply.

RequestInput

Additional undeclared properties: **rejected**.

RequestInput fields

Field	Type / required	Meaning / constraints
`leaseId`	string · Required	Authorized lease association; never accept a foreign tenant lease.
`category`	string · Required	Policy enum for policies; free-form string for clarification requests.
`subject`	string · Required	Clarification subject; enforce the schema length limit. Min length 1; Max length 120.
`message`	string · Required	Clarification body or safe validation detail, depending on model. Min length 1; Max length 5000.

Request

Additional undeclared properties: **rejected**.

Request fields

Field	Type / required	Meaning / constraints
`id`	string · Required	Opaque stable identifier. Authorize the underlying object independently of its format.
`tenantId`	string · Required	Owning tenant identity; derive/validate against authenticated scope.
`leaseId`	string · Required	Authorized lease association; never accept a foreign tenant lease.
`category`	string · Required	Policy enum for policies; free-form string for clarification requests.
`subject`	string · Required	Clarification subject; enforce the schema length limit.
`message`	string · Required	Clarification body or safe validation detail, depending on model.

COMPLETE DATA DICTIONARY

A6 / Clarification request workflow / continued

Request fields

Field	Type / required	Meaning / constraints
`status`	string · Required	Lifecycle/result state; see the model-specific enum and behavioral chapters. Allowed: open, answered.
`createdAt`	string · Required	Server creation timestamp with offset.
`reply`	string | null · Required	Landlord reply; null for an unanswered Request.

ReplyInput

Additional undeclared properties: **rejected**.

ReplyInput fields

Field	Type / required	Meaning / constraints
`reply`	string · Required	Landlord reply; null for an unanswered Request. Min length 1; Max length 5000.

COMPLETE DATA DICTIONARY

A7 / Compliance review metadata

Generated from the current OpenAPI schemas. Required means the key must be present; nullable means null is permitted as a value. Semantic/backend checks still apply.

Compliance

Additional undeclared properties: **rejected**.

Compliance fields

Field	Type / required	Meaning / constraints
`status`	string · Required	Lifecycle/result state; see the model-specific enum and behavioral chapters. Allowed: pending, passed, conflict, unavailable.
`state`	string | null · Optional	Jurisdiction code. NC is the only configured state in this contract.
`lawRevision`	string · Optional	Content-derived law snapshot revision. Backend must bind it to current applicable coverage.
`checkedAt`	string · Optional	Timestamp when this review was checked; must not be in the future.
`expiresAt`	string · Optional	Exclusive expiration instant for the review/source snapshot.
`contentHash`	string · Optional	SHA-256 lowercase hex of exact reviewed content; match canonical policy text or final answer as specified.
`asOf`	string · Optional	Requested/server business calendar date. Exact matching matters for review identity.
`kind`	string · Optional	Review target: policy or answer. Allowed: policy, answer.
`citations`	array<LawCitation> · Optional	Evidence references. Answer citations and legal-review citations are different models.
`findings`	array<object> · Optional	Review findings. A passed review cannot carry unresolved findings.
`scopeKey`	string · Optional	Exact review scope key; backend must also bind organization/object ownership.

> When status is passed, state, kind, scopeKey, asOf, checkedAt, expiresAt, lawRevision, contentHash and citations are all required. State must be NC, hashes must be 64 lowercase hex characters, citations cannot be empty, and findings must be absent/empty. Runtime checks additionally verify dates, scope, identity and source applicability.

COMPLETE DATA DICTIONARY

A7 / Compliance review metadata / continued

LawCitation

Additional undeclared properties: **allowed as defined by schema**.

LawCitation fields

Field	Type / required	Meaning / constraints
`url`	string · Required	Official law/guidance URL; current metadata allowlist is exact. Allowed: https://www.ncleg.gov/EnactedLegislation/Statutes/HTML/ByChapter/Chapter_42.html, https://ncdoj.gov/protecting-consumers/home-repair-and-products/renting-a-home/.
`section`	string · Required	Source section identifier; nonempty for legal-review citations. Min length 1.

COMPLETE DATA DICTIONARY

A8 / Errors and mutation acknowledgments

Generated from the current OpenAPI schemas. Required means the key must be present; nullable means null is permitted as a value. Semantic/backend checks still apply.

ErrorDetails

Additional undeclared properties: **rejected**.

ErrorDetails fields

Field	Type / required	Meaning / constraints
`code`	string · Required	Stable machine-readable error code; keep it safe and actionable.
`message`	string · Required	Clarification body or safe validation detail, depending on model.
`requestId`	string · Required	Server correlation ID safe to share for diagnostics.
`fields`	object · Required	Map of field names to safe human-readable validation messages. Additional values: string.

Error

Additional undeclared properties: **rejected**.

Error fields

Field	Type / required	Meaning / constraints
`error`	ErrorDetails · Required	Error envelope; no data property on failures.

Deleted

Additional undeclared properties: **rejected**.

Deleted fields

Field	Type / required	Meaning / constraints
`deleted`	true · Required	True on a successful staged-version cancellation. Constant: true.

COMPLETE DATA DICTIONARY

A8 / Errors and mutation acknowledgments / continued

Logout

Additional undeclared properties: **rejected**.

Logout fields

Field	Type / required	Meaning / constraints
`signedOut`	true · Required	True on a successful logout. Constant: true.

LoginInput

Additional undeclared properties: **rejected**.

LoginInput fields

Field	Type / required	Meaning / constraints
`email`	string · Required	Account email returned by the server; login accepts email/password only.
`password`	string · Required	Credential input only; never return or log it.

WORKED JSON RESPONSES

J / anonymousSession — Session

Complete Session example. Fictional identifiers, accounts and fixed timestamps are illustrative. Do not use these as production records or approvals. Generate fresh genuine review metadata for live checks.

Exact JSON response envelope


```
{
  "data": {
    "user": null,
    "csrfToken": null
  }
}
```

WORKED JSON RESPONSES

J / authenticatedSession — Session

Complete Session example. Fictional identifiers, accounts and fixed timestamps are illustrative. Do not use these as production records or approvals. Generate fresh genuine review metadata for live checks.

Exact JSON response envelope


```
{
  "data": {
    "user": {
      "id": "landlord-1",
      "name": "Alex Morgan",
      "email": "alex@alderliving.example",
      "role": "landlord",
      "organization": "Alder Living",
      "permissions": [
        "portfolio:read",
        "leases:read",
        "requests:write",
        "policies:write"
      ]
    },
    "csrfToken": "demo-only"
  }
}
```

WORKED JSON RESPONSES

J / registeredProperty — Property

Complete Property example. Fictional identifiers, accounts and fixed timestamps are illustrative. Do not use these as production records or approvals. Generate fresh genuine review metadata for live checks.

Exact JSON response envelope


```
{
  "data": {
    "id": "property-1",
    "name": "Willow House",
    "address": "128 Willow Lane",
    "city": "Raleigh, NC 27601",
    "state": "NC",
    "units": 5,
    "style": "willow",
    "description": "Quiet corners. Connected living.",
    "amenities": [
      "Pet friendly",
      "Bike storage",
      "Courtyard"
    ]
  }
}
```

WORKED JSON RESPONSES

J / emptyLandlordWorkspace — Workspace

Complete Workspace example. Fictional identifiers, accounts and fixed timestamps are illustrative. Do not use these as production records or approvals. Generate fresh genuine review metadata for live checks.

Exact JSON response envelope


```
{
  "data": {
    "asOf": "2026-10-31",
    "timeZone": "America/New_York",
    "user": {
      "id": "landlord-1",
      "name": "Alex Morgan",
      "email": "alex@alderliving.example",
      "role": "landlord",
      "organization": "Alder Living",
      "permissions": [
        "portfolio:read",
        "leases:read",
        "requests:write",
        "policies:write"
      ]
    },
    "properties": [],
    "leases": [],
    "tenants": [],
    "requests": [],
    "policies": [],
    "summary": {
      "properties": 0,
      "units": 0,
      "occupied": 0,
      "monthlyRent": 0,
      "outstandingRent": 0,
      "renewals": 0
    }
  }
}
```

WORKED JSON RESPONSES

J / pendingPolicy — PolicyVersion

Complete PolicyVersion example. Fictional identifiers, accounts and fixed timestamps are illustrative. Do not use these as production records or approvals. Generate fresh genuine review metadata for live checks.

Exact JSON response envelope


```
{
  "data": {
    "id": "document-example-v1",
    "familyId": "family-example",
    "title": "Maintenance policy",
    "name": "rules.pdf",
    "category": "maintenance",
    "propertyId": "property-example",
    "state": "NC",
    "status": "processing",
    "compliance": {
      "status": "pending",
      "state": "NC"
    },
    "version": 1,
    "effectiveFrom": "2026-11-15",
    "effectiveTo": null,
    "uploadedAt": "2026-10-31T12:00:00Z",
    "size": 2048,
    "pageCount": null,
    "uploadedBy": "Alex Morgan",
    "description": "Document processing and legal review are pending.",
    "excerpt": null
  }
}
```

WORKED JSON RESPONSES

J / clarificationAnswer — Answer

Complete Answer example. Fictional identifiers, accounts and fixed timestamps are illustrative. Do not use these as production records or approvals. Generate fresh genuine review metadata for live checks.

Exact JSON response envelope


```
{
  "data": {
    "id": "answer-example",
    "threadId": "thread-example",
    "answer": "The applicable policy is awaiting legal review. Please request a review before relying on its
terms.",
    "status": "needs_clarification",
    "citations": [],
    "calculations": [],
    "activity": [
      {
        "label": "Legal review is pending",
        "status": "attention"
      }
    ],
    "compliance": {
      "status": "pending",
      "state": "NC"
    }
  }
}
```

WORKED JSON RESPONSES

J / syntheticReviewedAnswer — Answer

Complete Answer example. Fictional identifiers, accounts and fixed timestamps are illustrative. Do not use these as production records or approvals. Generate fresh genuine review metadata for live checks.

Exact JSON response envelope


```
{
  "data": {
    "id": "answer-example",
    "threadId": "thread-example",
    "answer": "Illustrative reviewed response. Replace this fixture with an answer checked by your legal-review
service.",
    "status": "supported",
    "citations": [],
    "calculations": [],
    "activity": [
      {
        "label": "Illustrative fixture; no actual legal assessment performed",
        "status": "complete"
      }
    ],
    "demo": true,
    "compliance": {
      "status": "passed",
      "state": "NC",
      "kind": "answer",
      "scopeKey": "lease:lease-1",
      "asOf": "2026-10-31",
      "checkedAt": "2026-10-31T12:00:00Z",
      "expiresAt": "2026-11-01T12:00:00Z",
      "lawRevision": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
      "contentHash": "f06dc6f837f6376298027846ee624e183201299c515002c366a5533649e615c9",
      "citations": [
        {
          "url": "https://www.ncleg.gov/EnactedLegislation/Statutes/HTML/ByChapter/Chapter_42.html",
          "section": "Chapter 42 (illustrative reference only)"
        }
      ],
      "findings": []
    }
  }
}
```


This is a structural test fixture, not an actual legal assessment. Its lawRevision is a placeholder, its text is illustrative, and its fixed timestamps expire. The contentHash matches only the exact example answer string.

WORKED JSON RESPONSES

J / versionConflict — Error

Complete Error example. Fictional identifiers, accounts and fixed timestamps are illustrative. Do not use these as production records or approvals. Generate fresh genuine review metadata for live checks.

Exact JSON response envelope


```
{
  "error": {
    "code": "VERSION_CONFLICT",
    "message": "This policy changed. Refresh the library before adding a version.",
    "requestId": "example-request",
    "fields": {
      "expectedVersion": "The latest version is 3."
    }
  }
}
```

RELEASE REFERENCE

21 / References, version and handoff receipt

Contract snapshot

API contract version: 1.1.0. OpenAPI dialect: 3.1.0. Prepared October 31, 2026. All dates/IDs/credentials in worked examples are illustrative; generate current genuine review metadata and use your own authorized data.

SHA-256 of contracts/openapi.json used to generate this PDF

```
517b8eb2647ba722b73f5b7789075b1d12e8479bf3e4d60da9bd26e4897c0c6a
```

External references

- Vite environment replacement
- Vite development proxy
- Browser CORS and credential behavior
- Nginx upstream URI forwarding
- Configured NC General Assembly law source
- Configured NC Department of Justice guidance

Handoff receipt to complete

Record	Value to fill by the integration team
Backend release / contract version	_____
Development / staging / production API base	_____
Chosen browser topology and frontend origin	_____
Test account provisioning owner	_____
Ingestion / review / operational owner	_____
Explorer report and backend test evidence	_____
Application acceptance / release approval	_____

Regeneration and scope

This document is generated from authored integration guidance plus the repository's actual OpenAPI schemas and behavior. Rebuild it whenever the contract or adapter changes. The original repository retains editable source and a build manifest. The companion documentation ZIP contains this PDF, the OpenAPI file, example payloads and connection/behavioral guides.